



ADAPTER PATTERN

Kết nối hệ thống Order với thế giới bên ngoài

1. Mục tiêu bài học

Sau tuần này, sinh viên có thể:

- Hiểu bản chất Adapter (Phân biệt với Facade/Decorator)
- Nhận diện bài toán tích hợp hệ thống (3rd party SDK, Legacy API)
- Tách biệt Business Logic khỏi chi tiết kỹ thuật (DIP)
- Áp dụng Adapter cho:
 - Payment Gateway (PayPal, VNPay)
 - Shipping Provider (GHN, GHTK)
 - Legacy Systems

2. Đặt vấn đề – Tích hợp hệ thống ngoài

Hệ thống Order cần tích hợp:

- PayPal SDK (Method: MakePayment)
- VNPay SDK (Method: PayRequest)
- GHN API (Method: CreateShippingOrder)

🔗 Vấn đề:

1. Interface mỗi bên một kiểu.
2. Không thể sửa code của SDK (Bên thứ 3).
3. Nếu gọi trực tiếp trong OrderService -> Vi phạm DIP & OCP.

3. Ví dụ sai – Business phụ thuộc SDK

✗ Vi phạm Clean Architecture: Business Layer biết về External Lib.

- **Code này vi phạm Dependency Inversion Principle**

```
public class OrderService {  
    public void Checkout(decimal amount) {  
        // Trực tiếp phụ thuộc vào SDK cụ thể  
        var paypal = new PaypalSdk();  
  
        // Nếu mai một đổi sang VNPay -> Sửa  
        paypal.MakePayment((double)amount);  
    }  
}
```

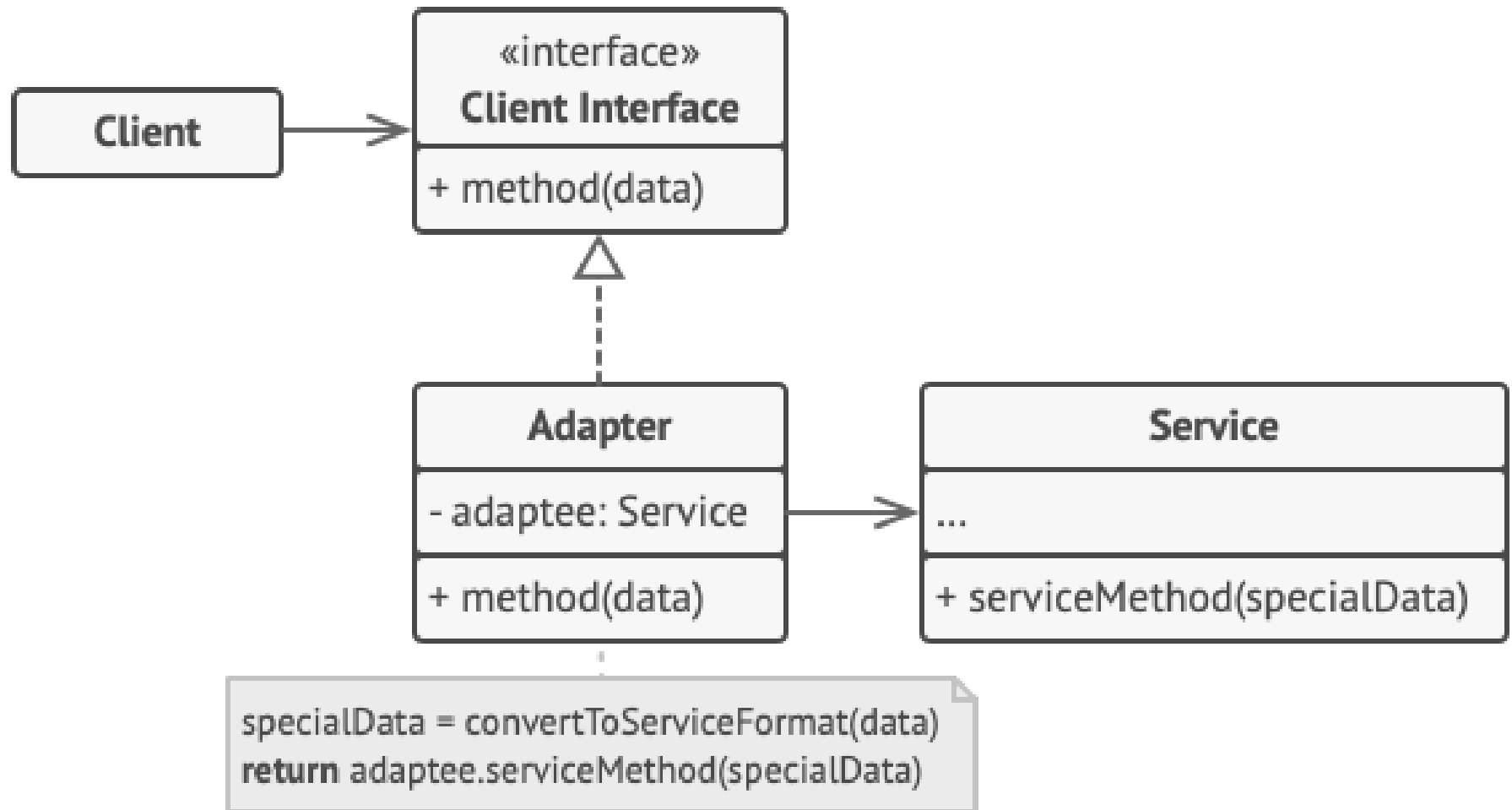
toàn bộ code này!

4. Ý tưởng cốt lõi Adapter

👉 Ý tưởng cốt lõi: Tạo một lớp trung gian (Adapter) để "phiên dịch" interface.

1. Client (OrderService): Chỉ nói chuyện với Interface chuẩn của mình.
2. Adapter: Nhận lệnh từ Client -> Chuyển đổi -> Gọi SDK thật.
3. Adaptee (SDK): Thực thi mà không biết mình đang được dùng qua Adapter.

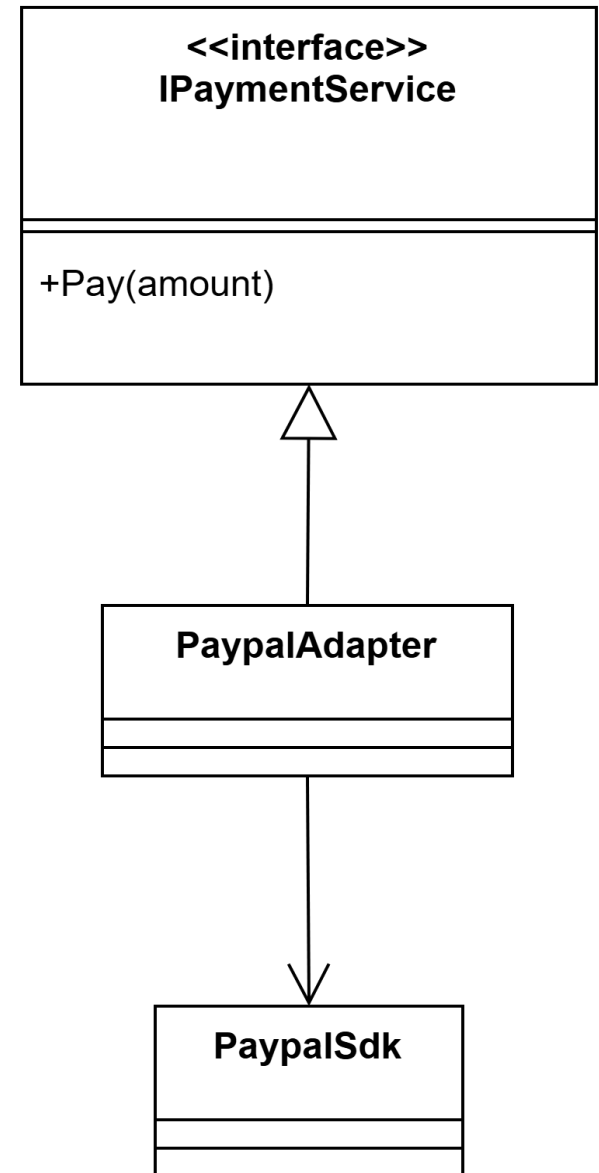
5. Structure of Adapter Pattern



5. Adapter Pattern in Order Management

📌 Business chỉ biết IPaymentService.

Adapter giấu đi sự phức tạp của SDK.



6. Thiết kế: Target & Adaptee

1. Interface của Business (Target)

```
public interface IPaymentService {  
    void Pay(decimal amount);  
}
```

2. SDK bên thứ 3 (Adaptee - Không sửa được)

```
public class PaypalSdk {  
    public void MakePayment(double money) {  
        Console.WriteLine($"PayPal charged:  
{money}");  
    }  
}
```


7. Implementation – The Adapter

3. Adapter: Nối Business với SDK

```
public class PaypalAdapter : IPaymentService {  
    private readonly PaypalSdk _paypal;  
  
    public PaypalAdapter(PaypalSdk paypal) {  
        _paypal = paypal;  
    }  
    public void Pay(decimal amount) {  
        // "Phiên dịch" tham số và tên hàm  
        // decimal -> double  
        // Pay -> MakePayment  
        _paypal.MakePayment((double)amount);  
    }  
}
```

8. OrderService (Clean Code)

```
public class OrderService {  
    private readonly IPaymentService _payment;  
    // Dependency Injection  
    public OrderService(IPaymentService payment) {  
        _payment = payment;  
    }  
    public void Checkout(decimal amount) {  
        // Chỉ biết Interface, không biết PayPal là gì  
        _payment.Pay(amount);  
    }  
}
```

9. Cách sử dụng (Wiring)

```
// Tại file cấu hình (Startup/Program.cs)  
var sdk = new PaypalSdk();
```

```
// Gói SDK vào Adapter  
IPaymentService payment = new PaypalAdapter(sdk);
```

```
// Inject Adapter vào Service  
var orderService = new OrderService(payment);
```

```
orderService.Checkout(1000);
```

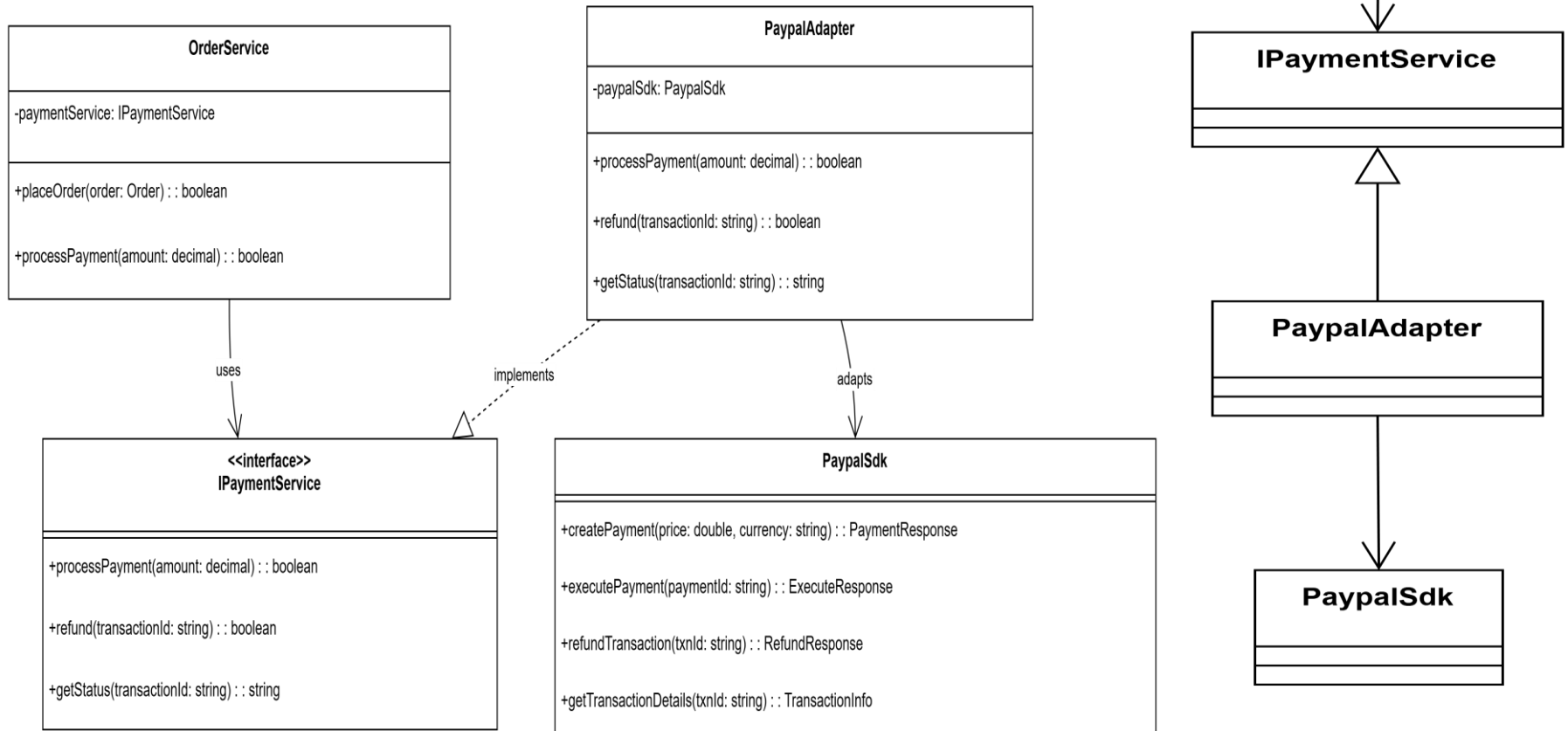
10. So sánh: Adapter vs Others

Pattern	Mục đích chính
Adapter	Chuyển đổi Interface (Kết nối 2 hệ thống khác nhau)
Strategy	Thay đổi thuật toán (Cùng 1 interface)
Decorator	Thêm hành vi/trách nhiệm (Không đổi interface)
Facade	Đơn giản hóa một hệ thống phức tạp

11. Adapter trong Clean Architecture

✚ Adapter nằm ở Infrastructure Layer.

Giúp bảo vệ Domain khỏi thay đổi bên ngoài.



12. Bài tập thực hành



Bài 1:

Tích hợp 'MomoSdk' (method: TransferMoney) vào hệ thống bằng Adapter.



Bài 2:

Viết Adapter cho 'GHNShippingService'.

Map dữ liệu từ 'Order' của ta sang 'ShippingRequest' của GHN.



Bài 3:

Vẽ sơ đồ lớp (UML) cho module Payment hỗ trợ cả PayPal, Momo, VNPay.

13. Kết luận

“ Adapter giúp hệ thống nói chuyện với thế giới mà không đánh mất bản sắc (kiến trúc) của mình. ”

“ Đừng để Business học cách dùng SDK.
Hãy để Adapter làm phiên dịch viên. ”